

- Application/use case: rocket telemetry, UAV, IoT, Wi-Fi, ground station, point-to-point, etc.
- Operating frequency or frequency range
- Required gain/link objective
- Bandwidth
- Polarization
- Available physical volume
- Directional or omnidirectional requirement
- Impedance/feed constraints
- Environment and mounting context
- Priority weights such as gain, size, bandwidth, efficiency, cost, simplicity

B. Recommendation

Convert the requirement into structured engineering constraints. Filter incompatible antenna families, score viable families, explain the recommendation, and show alternatives.

C. Antenna information page

After selection, show antenna theory, applications, limitations, design assumptions, construction method, and the requirement-specific design.

D. Design generation

Generate dimensions, parameterized geometry, feed assumptions, materials, and relevant electrical parameters. Every number must have provenance: equation, design rule, user input, or optimization.

E. Simulation

Show the fastest scientifically defensible analysis available. Clearly label calculated, predicted, simulated, and measured results.

F. Interactive optimization

Allow sliders/numeric edits to update the canonical design state, 3D geometry, and fast analysis. Provide parameter sweeps and optimization.

G. Export

Generate a professional datasheet with requirements, assumptions, dimensions, plots, metrics, model metadata, and revision history.

H. Learn/build/manufacture

Provide fabrication guidance, measurement guidance, student resources, and eventually a manufacturing quote/contact workflow.

3. PRODUCT DIFFERENTIATOR

The main differentiator is the **interactive requirement-to-design loop**. The platform connects requirements, antenna selection, equations, geometry, simulation, optimization, documentation, and manufacturing instead of forcing the user to switch tools.

Ideal user experience: “I tell the system what I need, it explains which antenna fits, generates a defensible design, shows predicted behavior, and lets me experiment without editing source code.”

4. LOOPHOLES / RISKS / SOLUTIONS

Risk / loophole	Why it is dangerous	Solution
Full-wave “live” simulation is slow	Repeated EM solves can be expensive and make the UI unusable.	Use three tiers: instant/fast preview, numerical analysis, and high-accuracy background jobs.
Users may confuse estimates with reality	Real antennas change with feed, material, mounting, ground plane and tolerance.	Separate Calculated / Predicted / Simulated / Measured and show assumptions.
One antenna formula is not universal	Different antenna families and feeding conditions require different models.	Use independent, versioned antenna models with validity ranges and assumptions.
Free-hand geometry is difficult	Arbitrary geometry requires topology, meshing, feed and solver support.	Start with parameterized editors; add constrained free geometry later.
Third-party antenna API dependency	The core project becomes opaque and dependent on another provider.	Own the engineering engine; use external libraries only for generic computation/rendering.
Recommendation may be wrong	Requirements can conflict or be incomplete.	Hard-constraint filtering + weighted scoring + explanation + alternatives.
Optimizer can create impractical designs	A mathematical optimum may violate manufacturing/mechanical limits.	Treat hard constraints as non-negotiable and include manufacturing penalties.
Compute cost can explode	Many users/sweeps can overload workers.	Caching, deterministic request hashes, queues, quotas, concurrency limits and adaptive sweeps.
Marketplace chicken-and-egg	Suppliers need demand and users need suppliers.	Start with Request Manufacturing Quote and a small verified supplier network.
Simulation may differ from manufactured antenna	Fabrication and installation affect RF behavior.	Add tolerance analysis and encourage physical validation.
AI can hallucinate numbers	LLM output is not a numerical engineering solver.	AI explains/orchestrates; deterministic engine supplies all numerical outputs.
Educational content rights	Videos/docs may not be reusable.	Create original material or use properly licensed resources.

Critical rule: never hide the boundary between an engineering model and physical reality. Trust and traceability are part of the product.

5. ENGINEERING TRUTH RULES

- Never present an estimate as a measured fact.
- Store model assumptions and validity ranges.
- Do not silently extrapolate beyond a model's validated range.
- Consider feed, material, mounting, ground plane, connector and manufacturing tolerance where relevant.
- If a defensible result cannot be produced, return a limitation/warning rather than a fabricated value.

- Engineering correctness has priority over visual polish.

6. SIMULATION STRATEGY

“Live simulation” should mean responsive interactive feedback, not a full-wave EM solve on every mouse movement.

Tier 1 — Fast preview

- Analytical or highly optimized model where scientifically justified
- Designed for interactive response
- Debounced slider updates
- Clearly labelled preview/prediction

Tier 2 — Numerical analysis

- More detailed numerical model
- Asynchronous when needed
- Cache identical requests
- Suitable for sweeps and exploration

Tier 3 — High-accuracy analysis

- Background worker/job
- Store geometry/model/settings metadata
- Show status/convergence where applicable
- Support retry/cancellation where practical

The AI must preserve this architecture. Do not “solve” performance by silently outsourcing antenna results to an external API.

7. ANTENNA MODEL INTERFACE

```
AntennaModel validate(requirements) calculate(parameters) generate_geometry(parameters) analyze(geometry, settings)
get_assumptions() get_validity_range() get_model_version()
```

Do not bury engineering formulas inside React components. The frontend is the interaction/visualization layer; the engineering engine is authoritative.

8. DATA CONTRACT

Requirement: application, frequency, bandwidth, gain_target, polarization, size_constraints, environment, priorities
Design: antenna_type, model_version, parameters, materials, feed, geometry, assumptions
SimulationResult: model_type, status, metrics, plots, warnings, computation_metadata
Revision: design_id, revision_number, changed_parameters, result_summary

Saved designs should remain reproducible from stored inputs and model version whenever practical.

9. RECOMMENDED TECH STACK

Layer	Recommended	Purpose
Frontend	Next.js + React + TypeScript	UI, routing, forms, application state
3D	Three.js / React Three Fiber	Interactive antenna geometry
Plots	Scientific plotting library	Radiation patterns, sweeps, frequency response

Layer	Recommended	Purpose
Backend	Python + FastAPI	Engineering API and orchestration
Numerical	NumPy + SciPy	Numerical computation and optimization
Database	PostgreSQL	Users, requirements, designs, revisions, metadata
Cache/Queue	Redis + worker system later	Caching and expensive jobs
Storage	Object storage later	Large simulation files and exports

10. CORE FEATURES

- Requirement wizard
- Explainable antenna recommendation
- Antenna knowledge pages
- Requirement-specific design calculator
- Parameterized 3D designer
- Fast radiation-pattern visualization
- Simulation dashboard
- Parameter sliders
- Undo/redo and design revisions
- Parameter sweeps
- Optimization
- Custom antenna mode
- Datasheet generation
- Student theory/build tutorials
- Measurement/S-parameter workflow later
- Manufacturing quote/contact workflow later

11. RECOMMENDATION ENGINE

Requirements ↓ Hard constraint filtering ↓ Candidate antenna families ↓ Weighted scoring ↓ Recommendation + alternatives ↓ Explanation

Recommendation must be deterministic and explainable. AI may turn structured results into natural language, but the score and engineering decision come from the rule/scoring layer.

12. CUSTOM ANTENNA MODE

The custom mode is for users who know their target performance. Inputs can include frequency, gain, bandwidth, size, polarization, impedance, environment and other constraints. The optimizer searches a valid design space.

Do not promise that arbitrary requirements are achievable. If constraints conflict, identify the conflict and show trade-offs. Return multiple candidate designs when useful, such as maximum-gain, compact, and wideband variants.

13. INTERACTIVE DESIGNER

Start with parameterized editing. For a Yagi, controls may include reflector length, driven-element length, each director length, element spacing, boom length, element radius, material, and feed settings.

UI event ↓ Validate input ↓ Update canonical design state ↓ Regenerate geometry ↓ Debounce ↓ Fast analysis ↓ Render metrics + plots ↓ Optional high-accuracy job

The canonical design state is the single source of truth for geometry and calculations.

14. FREE-HAND GEOMETRY — FUTURE

Free-hand editing is intentionally later. It requires robust geometry representation, topology rules, feeding definitions, meshing strategy, and solver compatibility. Do not implement it early merely because it looks impressive.

15. PARAMETER SWEEP

Example: Director length: 140–155 mm Step: 0.5 mm For each valid point: generate design analyze record metric Then: plot metric vs parameter identify candidates candidate → design revision

Use caching, deduplication and adaptive evaluation to control compute cost.

16. OPTIMIZATION

Optimization should support hard constraints and weighted objectives such as gain, bandwidth, efficiency, beamwidth, side-lobe level, front-to-back ratio, size, and manufacturing feasibility.

Start with single-objective optimization and small parameter spaces. Add multi-objective/Pareto optimization after the computation pipeline is stable.

The optimizer must never return a design that violates a declared hard constraint merely because its objective score is better.

17. MANUFACTURING WORKFLOW

- Initial implementation: Request Manufacturing Quote, not a full marketplace.
- Send frequency, dimensions, materials, connector/feed, quantity and tolerances.
- Pilot with a small verified supplier network.
- Only later add supplier profiles, pricing, lead times, ratings and commissions.

18. STUDENT LEARNING WORKFLOW

Each antenna should eventually have: Theory → Calculation → Design → Simulation → Fabrication → Measurement → Simulation-vs-Measurement.

Prefer original or properly licensed content. Teach validation rather than blind trust.

19. AI ASSISTANT RULES

- Parse natural-language requirements.
- Ask for missing requirements.
- Explain recommendation scores.
- Explain assumptions and plots.
- Generate summaries from verified data.
- Call deterministic calculation functions when tools exist.
- Never fabricate dimensions, gain, VSWR or simulation results.

- Never overwrite validated engineering outputs with an LLM guess.

20. DOMAIN OBJECTS

Object	Purpose
User	Identity, role, limits
Requirement	Structured engineering requirements
AntennaModel	Model, version, assumptions, validity
Design	Parameters, geometry, materials, feed
SimulationJob	Compute request/status/settings
SimulationResult	Metrics, plots, warnings, metadata
DesignRevision	Version history
ManufacturingRequest	Quote/contact workflow
LearningResource	Theory/build/measurement content

21. FRONTEND SCREENS

Screen	Purpose
Home	Explain product and start requirement flow
Requirement Wizard	Collect and validate requirements
Recommendation	Best antenna, alternatives, explanations
Antenna Page	Theory, limitations, assumptions, design
Design Workspace	3D geometry, controls, metrics
Simulation	Plots, metrics, model metadata
Optimization	Sweeps, candidates, comparisons
Datasheet	Export-ready report
Learn	Educational/build content
Manufacture	Quote/contact workflow

22. BACKEND MODULE MAP

backend/ api/ requirements recommendations designs simulations optimization exports engineering/ core/ units constants
validation geometry antennas/ model_1 model_2 model_3 analysis/ analytical numerical high_accuracy jobs/ database/ tests/

Exact names may evolve. Keep domain logic separate from API transport and UI.

23. TESTING STRATEGY

- Unit tests for unit conversion and equations.
- Reference-case tests for every antenna model.

- Geometry validity tests.
- Regression tests for known outputs.
- Property tests where scientifically appropriate.
- Integration tests for requirement → design → analysis.
- Critical frontend interaction tests.
- Performance tests for sweeps and concurrent jobs.

24. PERFORMANCE STRATEGY

- Debounce slider-driven calculations.
- Cache identical requests with a deterministic design hash.
- Do not recompute unchanged calculations.
- Use workers for expensive tasks.
- Limit concurrent high-cost jobs.
- Support cancellation/timeouts.
- Store large results appropriately.
- Show progress/status rather than freezing UI.

25. SECURITY & RELIABILITY

- Validate inputs server-side.
- Never execute arbitrary user code.
- Sandbox expensive workers.
- Authentication and authorization.
- Rate-limit expensive endpoints.
- Per-user compute quotas in production.
- Log failures without exposing secrets.
- Back up design/database data.
- Preserve reproducibility metadata.

26. STEP-BY-STEP DEVELOPMENT ROADMAP

Phase	Deliverable
0	Freeze scope, requirement schema, result terminology, model interface
1	Engineering core + first validated antenna models
2	FastAPI backend + database
3	Requirement wizard + recommendation UI
4	3D parameterized designer
5	Fast simulation + plots
6	Sweeps + design revisions

Phase	Deliverable
7	Optimization
8	High-accuracy numerical simulation workers
9	Datasheets + learning
10	Manufacturing quote workflow
11	Production hardening, monitoring, validation

27. MVP DEFINITION

- 3 well-understood antenna families.
- Requirement-to-recommendation workflow.
- Deterministic dimension calculation.
- 3D parameterized visualization.
- Fast radiation-pattern preview.
- Interactive parameter changes.
- At least one useful parameter sweep.
- Design revision history.
- Datasheet export.
- Basic learning/build content.

MVP success means a new user can start from a real requirement, receive a defensible recommendation, generate a design, understand assumptions, change a parameter, and observe the predicted response change.

28. DEVELOPMENT RULES FOR THE AI

- Inspect the existing repository before modifying it.
- Understand current architecture before introducing abstractions.
- Keep engineering formulas in the engineering layer.
- Do not rewrite working modules unnecessarily.
- Never hardcode outputs just to make a demo look correct.
- If a scientific model is not implemented, create an explicit interface/placeholder rather than fake results.
- If an assumption is uncertain, flag it for validation.
- Prefer small, testable vertical slices.
- Add reference tests after implementing engineering models.
- Keep units explicit and normalize internally.
- Preserve saved-design compatibility where practical.
- Version engineering models.

29. HOW THE AI SHOULD EXECUTE TASKS

When asked to implement a feature, follow this order:

- Understand requested user behavior.
- Identify affected domain objects and engineering models.
- Inspect existing code.
- Define/confirm data contract.
- Implement the smallest correct backend/engineering change.
- Add tests.
- Connect frontend.
- Build/test affected areas.
- Report changes, assumptions, limitations, and the next validation step.

For large work, break it into coherent vertical slices. Do not generate a huge untested code dump.

30. EXAMPLE — YAGI INTERACTIVE LOOP

Requirement: 865 MHz target gain directional size limit Recommendation: Yagi-Uda Design: calculate wavelength generate dimensions generate 3D geometry Preview: run fast radiation model User: changes Director 3 length System: validate update canonical parameters regenerate geometry debounce recompute fast analysis update pattern + metrics User: runs sweep System: evaluate range cache results plot response identify candidates User: selects candidate System: create revision allow high-accuracy analysis generate datasheet

31. DEFINITION OF DONE

A feature is not done merely because it renders. It is done when engineering logic is correct or explicitly bounded, inputs are validated, results are traceable, UI is connected to canonical state, tests exist for important logic, errors are handled, and the no-black-box-engineering rule is respected.

For simulation features, the model, assumptions, validity, computational method, and result type should be visible enough for a technically literate user to understand what they are seeing.

32. NORTH STAR

Tell us what you need ↓ Understand requirement ↓ Recommend antenna ↓ Generate defensible design ↓ 3D visualization ↓ Calculated / simulated behavior ↓ Interactive modification ↓ Sweep / optimize ↓ Validate ↓ Datasheet ↓ Learn / build / measure ↓ Manufacture

North-star principle: make antenna engineering accessible without making it opaque. Every engineering decision should remain editable, explainable, reproducible, and connected to analysis.

This document is the master context for AI-assisted development. Preserve the deterministic engineering engine, requirement-to-design workflow, accuracy terminology, and phased architecture unless the project owner explicitly changes the specification.